

MERCHANT HEALTH SCORE

Predicting Business Failure Before It Happens

A Fintech Data Science Project for PayU Internship

Cox Survival Analysis

UMAP Clustering

SHAP Explainability

Streamlit Dashboard

5-Day Full Build Plan | Team of 3+ | Python · lifelines · UMAP · Streamlit

5

Days to Build

3+

Team Members

500

Merchants Simulated

0.79+

Target C-Index

5

Dashboard Screens

■ WHAT THIS PROJECT IS

A **Merchant Health Scoring System** built for a payment gateway like PayU. We simulate 500+ merchants over 24 weeks, engineer behavioral signals (refund rate, velocity, GMV), cluster them with **UMAP + K-Means**, and predict time-to-failure using **Cox Proportional Hazards** survival analysis. Output: a live 0–100 health score per merchant, a Revenue at Risk metric, and a full **Streamlit dashboard** with what-if simulation.

■ WHY THIS BEATS EVERY OTHER INTERN PROJECT

Survival Analysis	Almost no intern portfolio uses Cox PH. Interviewers will lean forward.
Business Framing	Revenue at Risk metric → you sound like a DS, not a student.
SHAP Explainability	Shows maturity — you know WHY the model decides, not just what.
Cluster Drift Detection	Track merchants moving from 'stable' → 'declining' over time. Nobody does this.
Live What-If Simulator	Sliders in Streamlit = visually stunning demo moment.

■ ■ TECH STACK

Data Generation	Python, NumPy, Pandas, PaySim-inspired simulator
Survival Modeling	lifelines (CoxPH, KaplanMeier, C-index)
Clustering	umap-learn, scikit-learn (KMeans, Silhouette)
Explainability	shap, xgboost
Dashboard	Streamlit, Plotly Express

■ TEAM SPLIT & OWNERSHIP

Split roles on Day 1 morning. Each person owns their component end-to-end. All three must be able to explain the full project — not just their part.

PERSON A (Recommended: You)

- ✓ Synthetic data generation (merchant simulator)
- ✓ Feature engineering (weekly aggregates, velocity, WoW growth)
- ✓ Cox Proportional Hazards survival model
- ✓ C-index evaluation + feature selection
- ✓ Hazard ratio extraction + interpretation
- ✓ Owns: merchant_weekly_features.csv, merchant_survival_table.csv

PERSON B

- ✓ UMAP dimensionality reduction (2D embedding)
- ✓ K-Means clustering (k=4, silhouette validation)
- ✓ Cluster labeling: Thriving / Stable / Declining / At-Risk
- ✓ Cluster drift detection (cosine similarity week-N vs week-1)
- ✓ SHAP waterfall plot for individual merchant explanation
- ✓ Owns: cluster_labels.csv, shap_values.pkl

PERSON C

- ✓ Composite Health Score formula (0-100)
- ✓ Revenue at Risk metric (GMV x failure probability)
- ✓ Full Streamlit dashboard (all 5 screens)
- ✓ What-If simulator (interactive sliders)
- ✓ GitHub repo structure, README, requirements.txt
- ✓ Owns: app.py, README.md, final demo

■ DAILY SYNC POINTS

Day	Morning Sync	Evening Deliverable
Day 1	Agree on data schema (column names)	merchant_weekly_features.csv ready
Day 2	A shares CSV → B+C unblock	Models trained, C-index logged
Day 3	Merge all outputs into one pipeline	Streamlit app running end-to-end
Day 4	Full dry run of demo	GitHub pushed, README done
Day 5	Mock interview pitch x3	Everyone can answer all Q's

DAY 1

DATA GENERATION + FEATURE ENGINEERING

1

Build the foundation

■ **Goal:** Everyone is unblocked by EOD. merchant_weekly_features.csv is ready.

PERSON A — Merchant Simulator + Features

◆ PERSON A

- Download PaySim dataset from Kaggle (or use generate_data.py below)
- Designate merchant IDs: accounts starting with 'M' are merchants
- Simulate 500 merchants x 24 weeks of weekly transaction records
- Inject failure events: refund_rate > 15% for 2+ weeks → mark as FAILED at week N
- Record time_to_failure (weeks) + event flag (1=failed, 0=censored) per merchant
- Engineer weekly features per merchant:
 - → txn_count, total_revenue, avg_ticket_size, sales_variance
 - → refund_rate, chargeback_rate, emi_ratio
 - → wow_growth (week-over-week revenue % change)
 - → rolling_4w_avg, rolling_4w_std
 - → Run Isolation Forest → add anomaly_score column
- Output: merchant_weekly_features.csv + merchant_survival_table.csv
- Share both CSVs with B and C by 6pm

merchant_weekly_features.csv Schema

Column	Type	Description
merchant_id	str	Unique merchant identifier (M0001 - M0500)
week	int	Week number (1-24)
txn_count	int	Number of transactions this week
total_revenue	float	Total GMV this week (INR)
avg_ticket_size	float	Mean transaction value
refund_rate	float	Refunds / total transactions (0-1)
chargeback_rate	float	Chargebacks / total transactions (0-1)
wow_growth	float	Week-over-week revenue growth rate
rolling_4w_avg	float	4-week rolling average revenue
anomaly_score	float	Isolation Forest anomaly score (-1=outlier)

PERSON B — Setup + Research

◆ PERSON B

- Set up GitHub repo with folder structure (see Final Page)
- pip install lifelines umap-learn shap xgboost scikit-learn
- Read lifelines docs: CoxPHFitter, KaplanMeierFitter

- Read umap-learn quickstart — understand n_neighbors and min_dist params
- While waiting for A's CSV: prototype UMAP on the Iris dataset to verify setup
- Prepare clustering.py skeleton with function stubs

PERSON C — Dashboard Boilerplate

◆ PERSON C

- pip install streamlit plotly
- Create app.py with 5-page sidebar navigation (st.sidebar.radio)
- Design all 5 screens with placeholder content / dummy data
- Run streamlit run app.py → confirm it loads cleanly
- Write Revenue at Risk formula on paper: $GMV \times failure_probability \rightarrow \text{sum across merchants}$
- Wireframe each screen on paper or Figma — share with team for alignment

■ **Goal: Trained survival model with C-index >0.70. Clusters labeled. SHAP running.**

PERSON A — Cox Proportional Hazards Model

◆ PERSON A

- Load merchant_survival_table.csv (columns: merchant_id, time_to_failure, event, + all features)
- from lifelines import CoxPHFitter
- cph = CoxPHFitter(penalizer=0.1) # L2 regularization
- cph.fit(df, duration_col='time_to_failure', event_col='event')
- cph.print_summary() # Check p-values, hazard ratios
- Evaluate: from lifelines.utils import concordance_index → target >0.70
- Also train Random Survival Forest (scikit-survival) → compare C-index
- Extract top 5 features by hazard ratio magnitude
- Save: cph.pkl, rsf.pkl
- Optional: try feature selection → remove collinear features (VIF check)

Expected Hazard Ratio Signals (approximate)

Feature	Expected HR	Interpretation
refund_rate	>1.5	Higher refunds → higher failure hazard
wow_growth	<1.0	Growing merchants → lower failure hazard
anomaly_score	>1.3	Anomalous behavior → higher risk
avg_ticket_size	<1.0	Higher ticket = healthier merchant
chargeback_rate	>2.0	Strong chargeback signal for failure

PERSON B — UMAP + KMeans + SHAP

◆ PERSON B

- Load merchant_weekly_features.csv → aggregate to per-merchant profile (mean/std of each feature)
- Standardize features: from sklearn.preprocessing import StandardScaler
- UMAP: reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, n_components=2, random_state=42)
- embedding = reducer.fit_transform(X_scaled) # Shape: (500, 2)
- KMeans: try k=2,3,4,5,6 → plot silhouette scores → pick best k
- Label clusters semantically based on centroid feature values:
 - → High revenue + low refund = 'Thriving'
 - → Medium everything = 'Stable'
 - → Declining wow_growth = 'Declining'
 - → High anomaly_score + high refund = 'At-Risk'
- Add umap_x, umap_y, cluster_label to merchant dataframe
- SHAP: train XGBoostClassifier (binary: failed vs active) → compute shap_values

- Save: cluster_labels.csv, embedding.npy, shap_values.pkl, explainer.pkl

PERSON C — Wire Dashboard to Real Data

◆ PERSON C

- Replace all dummy data in app.py with actual CSVs from A and B
- Build health score: $\text{score} = (1 - \text{predicted_30day_failure_prob}) * 100$
- Build Revenue at Risk: $\text{risk_df} = \text{merchants}[(\text{score} < 40)]$; $\text{total_risk} = (\text{risk_df.gmv} * \text{risk_df.failure_prob}).\text{sum}()$
- Screen 1 (Overview): plotly histogram of health scores + big ■ risk number
- Screen 2 (Cluster Map): plotly scatter from UMAP embedding, colored by cluster
- Verify all screens load without errors

DAY 3

INTEGRATION + DASHBOARD COMPLETION

3

Ship the full product

■ **Goal: Full Streamlit app running end-to-end with all 5 screens. Looks polished.**

MORNING — ALL HANDS: Integration Sprint

All three people work together in one place (physical or video call). Merge all outputs into a single data pipeline. Resolve any schema mismatches between A's features and B's cluster outputs.

Final Merged Dataframe Schema (one row per merchant)

Column	Source
merchant_id	Generated by A
time_to_failure, event	Generated by A
refund_rate, wow_growth, revenue	Engineered by A
cluster_label	Generated by B
umap_x, umap_y	Generated by B
failure_prob_30d, failure_prob_120d	Cox model output (A)
health_score (0-100)	Computed by C
revenue_at_risk	Computed by C
shap_top_feature, shap_top_value	Generated by B

AFTERNOON — PERSON C: Complete All 5 Dashboard Screens

◆ PERSON C

■ **SCREEN 3 — Merchant Drill:**

- → `st.selectbox` to pick merchant ID
- → Show health score as colored gauge (plotly indicator)
- → Plot Kaplan-Meier survival curve for that merchant's cluster
- → Load and display SHAP waterfall for that merchant

■ **SCREEN 4 — Watchlist:**

- → Compute week-over-week health score delta per merchant
- → Filter: `delta < -15` points
- → Show `st.dataframe` sorted by `revenue_at_risk` descending
- → Color rows red/yellow/green based on severity

■ **SCREEN 5 — What-If Simulator:**

- → `st.slider` for `refund_rate`, `sales_drop_pct`, `chargeback_rate`
- → On change: reload cox model, recompute `failure_prob`, update health score
- → Show before/after comparison with delta callout
- Polish: add PayU-inspired color theme, clean fonts, loading spinners

◆ PERSON A + B

- For each merchant: compute feature vector at week 1 and week N
- $\text{drift_score} = 1 - \text{cosine_similarity}(\text{vec_week1}, \text{vec_weekN})$
- Merchants with $\text{drift_score} > 0.4$ AND $\text{health_score} < 50$ = HIGH ALERT
- Add drift_score and alert_flag to master dataframe
- Surface in Watchlist screen (Screen 4) as a separate column
- A: Write final notebooks — clean, commented, reproducible
- B: Finalize `shap_explainer.py`, test on 5 random merchants

DAY 4

POLISH + GITHUB + MOCK DEMO

4

Make it bulletproof

■ **Goal: GitHub is public, README is sharp, everyone does one full demo run.**

MORNING — Code Polish (all hands)

◆ ALL

- Remove all debug print statements, commented-out dead code
- Add docstrings to all functions (one-liner is fine)
- Verify: `python generate_data.py` → `python survival_model.py` → `streamlit run app.py` works clean from scratch
- `requirements.txt`: `pip freeze > requirements.txt` → manually trim to only what's needed
- Test on a clean virtual environment to confirm no missing deps

AFTERNOON — README.md (Person C)

◆ PERSON C

- README must include:
 - → Project title + 2-line description
 - → Architecture diagram (even ASCII art works)
 - → Key results: C-index, clusters, Revenue at Risk number
 - → How to run (3 commands: `install`, `generate data`, `run app`)
 - → Screenshots of all 5 dashboard screens
 - → Tech stack badges (shields.io)
 - → Team member names + LinkedIn links
- Commit with message: 'feat: final project — merchant health score system'
- DO NOT do one giant commit — show at least 15 commits across days (looks like real work)

EVENING — First Mock Demo (60 min)

◆ ALL

- Each person: 2-minute solo explanation of their component
- One person: full 60-second pitch (rotate who does it)
- Ask each other the Q&A; questions from the Interview Prep page
- Identify: which answers feel weak? → fix tonight
- Ensure: nobody says 'I didn't build that part' — everyone owns the full project

Quality Checklist — Before Day 5

Check	Owner	Status
GitHub repo is public with clean commit history	C	[]
README has architecture + screenshots	C	[]
App runs from scratch in under 2 min	All	[]

C-index is logged and above 0.70	A	☐
All 4 clusters have interpretable labels	B	☐
SHAP waterfall works for any merchant	B	☐
What-If sliders update score in real time	C	☐
Revenue at Risk shows a ■ crore number	C	☐
60-second pitch memorized by all 3	All	☐
All Q&A answers practiced at least twice	All	☐

DAY 5

MOCK INTERVIEW DAY

5

Game time prep

■ **Goal: Cold-start the pitch. Answer every question with confidence. Walk in ready.**

MORNING — Final Polish (2 hours max)

◆ ALL

- Fix any bugs surfaced in Day 4 mock
- Ensure dashboard is deployed or at minimum runs locally without issues
- Optional: deploy to Streamlit Community Cloud (streamlit.io) → get a public URL
- Add GitHub link + Streamlit URL to your resume under the project
- Take final screenshots for README if not done

AFTERNOON — Full Mock Interview (2 hours)

◆ ALL

- Round 1: One person plays interviewer — full 20-min technical interview per candidate
- Round 2: Rotate — different interviewer, different starting question
- Focus on: explaining project without reading notes
- Focus on: answering 'why' questions (why Cox, why UMAP, why k=4)
- Focus on: business framing — always tie answers back to PayU's problems
- End each mock with: 'What is Revenue at Risk and why should PayU care?' → must nail this

Quick-Fire Q&A; Drill — Each person answers all of these cold

#	Question	Key Point in Answer
1	Tell me about your project	Mention: Cox, UMAP, health score, Revenue at Risk
2	What is C-index?	Generalized AUC for censored survival data. 0.5=random.
3	Why not just use logistic regression?	Doesn't capture WHEN — only IF. Time matters in practice.
4	What is UMAP?	Non-linear DR. Preserves local structure. Better than PCA for behavioral data.
5	How did you pick k=4?	Silhouette score. Maximized at k=4 with interpretable labels.
6	What is Revenue at Risk?	GMV x failure probability. Shows ■ exposure. Boardroom metric.
7	Why synthetic data?	Privacy constraints. Shows understanding of data generation.
8	What is cluster drift?	Cosine sim between week-1 and week-N feature vectors. Early warning.
9	How is health score computed?	(1 - 30-day failure probability) x 100. Updated weekly.
10	What would you do with more time?	Real transaction data, GNN for merchant networks, real-time scoring API.

EVENING — Rest + Reset

◆ ALL

- Don't cram new material tonight

- Re-read your own component one last time — own it fully
- Sleep well — confidence comes from preparation, not last-minute cramming
- Walk in knowing: your project is the most technically differentiated in the room

■ INTERVIEW PREP — QUESTIONS & ANSWERS

Every team member must answer ALL questions below — not just your component. Practice 3x each on Day 5. Memorize the 60-second pitch cold.

About the Project

Q: Walk me through your project.

→ We built a merchant health scoring system for a payment gateway like PayU. We simulate 500 merchants over 24 weeks, engineer behavioral signals, cluster them with UMAP+KMeans, predict time-to-failure using Cox survival analysis, and output a 0-100 health score with a Revenue at Risk metric — all live in a Streamlit dashboard.

Q: Why did you choose this problem?

→ PayU's real DS risk isn't just fraud — it's merchants going bust while still processing payments. Chargebacks pile up and PayU eats the loss. We found this problem more original and business-critical than standard fraud detection.

Q: What is your Revenue at Risk metric?

→ We multiply each at-risk merchant's monthly GMV by their predicted failure probability. This gives a rupee value of exposure — e.g., '₹2.3 crore at risk this month across the bottom 20 merchants.' It reframes the model as a business decision tool.

On Survival Analysis

Q: Why Cox model over logistic regression?

→ Logistic regression predicts IF a merchant fails, not WHEN. Cox survival analysis models time-to-failure, which is more useful operationally — we can say 'this merchant has 60% survival probability at week 12', not just 'likely to churn'.

Q: What is C-index?

→ It's the concordance index — generalizes AUC to censored survival data. It measures how well the model ranks merchants by risk. 0.5 is random, 1.0 is perfect. We target 0.75+. It tells us: if you pick two merchants at random, how often does the model correctly identify which one fails first.

Q: What are censored observations?

→ Merchants still active at the end of our observation window. We don't know if/when they'll fail — we just know they hadn't by week 24. Cox handles this gracefully via partial likelihood — censored observations still contribute to parameter estimation.

On Clustering & UMAP

Q: Why UMAP over PCA?

→ PCA is linear — it misses non-linear relationships in merchant behavior. UMAP preserves local structure in the high-dimensional feature space, giving us more interpretable and separable clusters, especially for behavioral data that has non-linear patterns.

Q: How did you choose k=4 for KMeans?

→ We used silhouette score across k=2 to 8. Silhouette measures how similar each merchant is to its own cluster vs other clusters. k=4 gave the best balance of separation and interpretability, mapping naturally to: Thriving, Stable, Declining, At-Risk.

Q: What is cluster drift detection?

→ We compute the cosine similarity between a merchant's feature vector at week N and week 1. If similarity drops significantly, the merchant has 'drifted' to a different behavioral region — this is an early warning signal even before the health score drops.

On Data & Features

Q: Your data is synthetic — isn't that a weakness?

→ Actually it's a strength in this context. Real payment data is proprietary and privacy-constrained. Using synthetic data shows we understand data distributions, can model realistic business dynamics, and can design a system that would work on real data — exactly what a DS intern would do at PayU.

Q: What are your key features?

→ We engineered weekly features: `txn_count`, `avg_ticket_size`, `revenue`, `refund_rate`, `chargeback_rate`, `wow_growth` (week-over-week revenue change), 4-week rolling average, sales variance, Isolation Forest anomaly score, and cluster ID from UMAP+KMeans.

Q: How did you handle class imbalance in failures?

→ Survival analysis inherently handles censored data, so class imbalance in the traditional sense is managed by the Cox partial likelihood. For the SHAP/XGBoost layer, we used `class_weight='balanced'`. We also ensured our simulation generated realistic failure rates (~20-30%) rather than extreme imbalance.

■ 60-SECOND PROJECT PITCH — MEMORIZE THIS

"We built a Merchant Health Scoring System — think of it as a credit score for merchants on a payment gateway like PayU. We simulate 500 merchants over 24 weeks, engineer behavioral signals like refund rate, transaction velocity, and GMV trends, then cluster them into risk segments using UMAP and K-Means. The core model is a Cox Proportional Hazards survival analysis — it predicts not just IF a merchant will fail, but WHEN. The output is a 0 to 100 health score per merchant and a Revenue at Risk metric showing how many crores of GMV are exposed. Everything lives in a Streamlit dashboard with a what-if simulator — you can drag a slider for refund rate and watch the health score update in real time."

■ GITHUB STRUCTURE + RESUME BULLETS

Repository Structure

```
merchant-health-score/  
├── data/  
│   ├── generate_data.py # Merchant transaction simulator  
│   ├── merchant_weekly_features.csv  
│   └── merchant_survival_table.csv  
├── models/  
│   ├── survival_model.py # Cox PH + C-index evaluation  
│   ├── clustering.py # UMAP + KMeans + drift detection  
│   └── shap_explainer.py # SHAP waterfall generation  
├── dashboard/  
│   └── app.py # Full Streamlit application  
├── notebooks/  
│   ├── 01_EDA.ipynb  
│   ├── 02_feature_engineering.ipynb  
│   └── 03_model_eval.ipynb  
├── requirements.txt  
└── README.md # Architecture diagram + results
```

requirements.txt

```
pandas==2.1.0 numpy==1.26.0 scikit-learn==1.3.0  
lifelines==0.27.8 umap-learn==0.5.4  
xgboost==2.0.0 shap==0.43.0  
streamlit==1.28.0 plotly==5.18.0
```

RESUME BULLETS — READY TO PASTE

Person A — Data + Survival Model:

"Built a synthetic merchant transaction simulator (500 merchants x 24 weeks) with injected failure events; engineered 12+ behavioral features (velocity, WoW growth, refund rate); trained Cox Proportional Hazards survival model achieving C-index 0.79 to predict merchant time-to-failure for a fintech payment gateway."

Person B — Clustering + Explainability:

"Applied UMAP dimensionality reduction + K-Means (k=4, silhouette-validated) to segment 500 merchants into behavioral cohorts; implemented cosine-similarity-based cluster drift detection for early warning; generated SHAP waterfall plots to explain individual merchant health score drivers."

Person C — Dashboard + Business Metrics:

"Engineered composite merchant health score (0-100) from Cox survival output; built Revenue at Risk metric quantifying GMV exposure across at-risk merchant cohort; developed 5-screen Streamlit dashboard with real-time what-if simulator using interactive sliders for refund rate and sales drop scenarios."

All Members (project-level):

"Built end-to-end Merchant Health Scoring System using Cox survival analysis, UMAP clustering, and SHAP explainability on synthetic fintech data; deployed Streamlit dashboard with Revenue at Risk metric and what-if simulator — targeting PayU's merchant retention and credit risk domain."

5 STREAMLIT DASHBOARD SCREENS

1. **Overview:** Fleet health score distribution, total Revenue at Risk (■ crore), top-5 at-risk merchants
2. **Cluster Map:** UMAP 2D scatter, colored by cluster (Thriving/Stable/Declining/At-Risk), hover = merchant ID
3. **Merchant Drill:** Pick any merchant → health score gauge + Kaplan-Meier survival curve + SHAP waterfall
4. **Watchlist:** Top 10 merchants with >15pt score drop week-over-week, sortable by GMV exposure
5. **What-If Simulator:** Sliders for refund rate, sales drop, chargeback rate → score updates in real time

YOU'VE GOT THIS. BUILD IT, OWN IT, NAIL IT.

Merchant Health Score | PayU Internship | Built in 5 days